

Московский авиационный институт
(Национальный исследовательский университет)
Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №3
по курсу «Теория сложности алгоритмов»
Вариант №2

Выполнила студентка: Бондарева Е. Е.

Группа: М8О-109СВ-25

Преподаватель: Рассказова В. А.

Дата: ____07.04.2026____

Оценка: _____

Москва, 2026

Задание

Для заданного языка, в котором предполагается, что ДКА определён над алфавитом $\{0, 1\}$:

1. построить описание МТ, решающей его;
2. реализовать данную МТ в виде программы на любом языке программирования.

$A = \{\langle M \rangle: M - \text{ДКА, который допускает хотя бы одну строку чётной длины}\};$

Теоретическая часть

Опр. 3.1. Детерминированной МТ (ДМТ) называется семерка $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, где

Q -- множество состояний МТ,

Σ -- входной алфавит,

Γ -- алфавит ленты такой, что $\Sigma \cup \{ \sqcup \} \subset \Gamma$,

$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times L, R$ -- переходная функция,

q_0 -- начальное состояние,

q_{accept} -- допускающее состояние,

$q_{\text{reject}} \in Q \setminus q_{\text{accept}}$ -- отвергающее состояние.

Опр. 3.2. Конфигурацией МТ $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ называется совокупность текущего содержимого ленты, состояния и положения пишущего устройства. Для $u, v \in \Gamma^*$ и $q \in Q$ через uqv обозначается конфигурация МТ, находящейся в состоянии q , у которой на ленте написано uv и пишущее устройство находится на первом символе v .

Опр. 3.3. Начальной конфигурацией МТ на строке $\omega \in \Sigma^*$ называется конфигурация $q_0\omega$.

Опр. 3.4. Допускающей или отвергающей конфигурациями МТМ $= (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ называется любая конфигурация uqv такая, что $q = q_{\text{accept}}$ или $q = q_{\text{reject}}$ соответственно.

Опр. 3.5. Говорят, что для МТ $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, $a, b, c \in \Gamma$, $u, v \in \Gamma^*$ и $q_i, q_j \in Q$

- 1) конфигурация $uq_i b v$ порождает конфигурацию $uq_j a c v$ если $\delta(q_i, b) = q_j, c, L$,
- 2) конфигурация $uq_i b v$ порождает конфигурацию $u a c q_j v$ если $\delta(q_i, b) = q_j, c, R$

Опр. 3.6. Говорят, что $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ допускает (или отвергает) строку $\omega \in \Sigma^*$, если существует последовательность конфигураций $C_1, \dots, C_k$ такая, что

- 1) C_1 -- начальная конфигурация,
- 2) C_i порождает C_{i+1} для всех i от 1 до $k-1$,
- 3) C_k -- допускающая (или отвергающая) конфигурация.

Опр. 3.7. Множество всех строк, которые допускает МТМ называется языком МТ $L(M)$. Такие языки называются распознаваемыми по Тьюрингу (или просто распознаваемыми). Иными словами, формальный язык называется распознаваемым, если существует ДМТ, которая его распознает.

Замечание 3.1. В некоторых источниках язык $L(M)$ называется полурешимым или частично решимым.

Замечание 3.2. Согласно принципам работы МТ в отличие от конечных автоматов такая модель если не отвергает строку, то из этого не следует, что она ее допускает. Некоторые конфигурации могут циклически порождать самих себя (по сути МТ может заиклиться на данном входе).

Многоленточная МТ (ММТ). Предполагается, что у данной МТ несколько рабочих лент и пишущих устройств. При этом вычисления производятся одновременно на всех лентах. Основное отличие от ДТМ состоит в определении переходной функции:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$$

где k -- число рабочих лент.

Теорема 3.1. Для любой ММТ существует эквивалентная ей ДМТ.

Недетерминированная МТ (НМТ). Предполагается, что данная МТ производит недетерминированные вычисления, то есть допускает переход из текущей конфигурации в некоторое конечное множество производных конфигураций. Основное отличие от ДТМ состоит в определении переходной функции:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$$

Теорема 3.2. Для любой НМТ существует эквивалентная ей ДМТ.

Следствие 3.1. Формальный язык A является распознаваемым тогда и только тогда, когда существует ДМТ или ММТ или НМТ, которая его распознает.

Опр. 3.8. МТ, которая либо допускает, либо отвергает входную строку (то есть всегда останавливается и не закликивается), называется решателем. А соответствующий язык МТ -- разрешимым по Тьюрингу (или просто разрешимым).

Следствие 3.2. Любой разрешимый язык является распознаваемым. А также, из примера 3.1., любой КС-язык является разрешимым.

Код программы

```
class DFA:
    def __init__(self, states, alphabet, transitions,
start_state, accept_states):
        self.states = states
        self.alphabet = alphabet
        self.transitions = transitions # dict: (state, symbol)
-> next_state
        self.start_state = start_state
        self.accept_states = accept_states

def turing_machine_simulation(dfa_encoding):
    """
    Симуляция МТ, проверяющей условие языка А
    """
    # Парсинг кодировки ДКА (упрощённая версия)
    dfa = parse_dfa_encoding(dfa_encoding)

    # Шаг 1: Проверяем, есть ли строка чётной длины, которую
    принимает ДКА
    # Используем алгоритм поиска в ширину для проверки
    достижимости

    # Строим граф достижимости для чётных длин
    # Состояние: (текущее_состояние_ДКА, чётность_длины)
    # чётность_длины: 0 - чётная, 1 - нечётная

    visited = dict()
    queue = [(dfa.start_state, 0)] # Начинаем с пустой строки
    (длина 0 - чётная)
    visited[(dfa.start_state, 0)] = ['']

    while queue:
        current_state, parity = queue.pop(0)

        # Если достигли принимающего состояния с чётной длиной
        if current_state in dfa.accept_states and parity == 0:
            print("Подходящая строка: ",
''.join(visited[(current_state, parity)]) or "Пустая строка")
            return True # МТ переходит в q_аccept

        # Переходы по всем символам алфавита
        for symbol in dfa.alphabet:
            next_state = dfa.transitions.get((current_state,
symbol))

            if next_state is not None:
                new_parity = 1 - parity # Меняем чётность
                new_state = (next_state, new_parity)
```

```

        if new_state not in visited:
            visited[new_state] = visited[(current_state,
parity)] + [symbol]
            queue.append(new_state)

    return False    # MT переходит в q_reject

def parse_dfa_encoding(encoding):
    """
    Упрощённый парсер кодировки ДКА
    Формат: "Q={q0,q1,q2};Σ={0,1};δ={(q0,0)->q1,(q0,1)-
>q0,...};q0;F={q1}"
    """
    # Разбираем кодировку
    parts = encoding.split(';')

    # Извлекаем состояния
    states_str = parts[0][2:]    # Убираем "Q="
    states = set(states_str.strip('{}').split(','))

    # Извлекаем алфавит
    alphabet_str = parts[1][2:]    # Убираем "Σ="
    alphabet = set(alphabet_str.strip('{}').split(','))

    # Извлекаем переходы
    transitions_str = parts[2][2:]    # Убираем "δ="
    transitions = {}
    transitions_parts = transitions_str.strip('{}').split(', ')

    for trans in transitions_parts:
        if not trans:
            continue
        trans = trans.strip('()')
        left, right = trans.split('->')
        state, symbol = left.split(',')
        state = state.strip()
        symbol = symbol.strip()
        transitions[(state, symbol)] = right.strip()

    # Стартовое состояние
    start_state = parts[3].strip()

    # Принимающие состояния
    accept_str = parts[4][2:]    # Убираем "F="
    accept_states = set(accept_str.strip('{}').split(','))

    return DFA(states, alphabet, transitions, start_state,
accept_states)

```

```

# Тестирование
def turing_machine():
    dfa_encoding = [
        {
            "dfa": "Q={q0,q1};Σ={0,1};δ={(q0,0)->q1, (q0,1)->q1, (q1,0)->q0, (q1,1)->q0};q0;F={q0}",
            "description": "ДКА, который принимает строки чётной длины",
            "expected": True,
        },
        {
            "dfa": "Q={q0,q1};Σ={0,1};δ={(q0,0)->q1, (q0,1)->q1, (q1,0)->q0, (q1,1)->q0};q0;F={q1}",
            "description": "ДКА, который принимает только строки нечётной длины",
            "expected": False,
        },
        {
            "dfa": "Q={q0};Σ={0,1};δ={(q0,0)->q0, (q0,1)->q0};q0;F={q0}",
            "description": "ДКА, который принимает все строки",
            "expected": True,
        },
        {
            "dfa": "Q={q0};Σ={0,1};δ={(q0,0)->q0, (q0,1)->q0};q0;F={}",
            "description": "ДКА, который не принимает ничего",
            "expected": False,
        },
        {
            "dfa": "Q={q0,q1};Σ={0,1};δ={(q0,0)->q0, (q0,1)->q1, (q1,0)->q1, (q1,1)->q0};q0;F={q1}",
            "description": "ДКА, который принимает строки с нечётным количеством единиц",
            "expected": True, # Принимает строки чётной длины?
        },
        {
            "dfa": "Q={q0,q1,q2,q3,q4,q5,q6};Σ={0,1};δ={(q0,0)->q1, (q0,1)->q1, (q1,0)->q2, (q1,1)->q2, (q2,0)->q3, (q2,1)->q3, (q3,0)->q4, (q3,1)->q4, (q4,0)->q6, (q4,1)->q5, (q5,0)->q5, (q5,1)->q5, (q6,0)->q6, (q6,1)->q6};q0;F={q5}",
            "description": "ДКА, который принимает строки с 1 на 5й позиции",
            "expected": True, # Проверяем:
        },
        {
            "dfa": "Q={q0,q1,q2,q3,q4,q5};Σ={0,1};δ={(q0,0)->q1, (q0,1)->q1, (q1,0)->q2, (q1,1)->q2, (q2,0)->q3, (q2,1)->q3, (q3,0)->q5, (q3,1)->q4, (q4,0)->q4, (q4,1)->q4, (q5,0)->q5, (q5,1)->q5};q0;F={q4}",

```

```

        "description": "ДКА, который принимает строки с 1 на
4й позиции",
        "expected": True,
    }
]

for test_number in range(len(dfa_encoding)):
    print("-" * 20)

    example = dfa_encoding[test_number]
    dfa = example["dfa"]
    print(f"ДКА: {dfa}")
    description = example["description"]
    expected = example["expected"]

    print(description)
    print(f"Ожидается: {expected}")

    result = turing_machine_simulation(dfa)

    print(f"Получено: {result}")

if __name__ == "__main__":
    turing_machine()

```

Тесты

```

-----
ДКА: Q={q0,q1};Σ={0,1};δ={(q0,0)->q1, (q0,1)->q1, (q1,0)->q0,
(q1,1)->q0};q0;F={q0}
ДКА, который принимает строки чётной длины
Ожидается: True
Подходящая строка: Пустая строка
Получено: True
-----
ДКА: Q={q0,q1};Σ={0,1};δ={(q0,0)->q1, (q0,1)->q1, (q1,0)->q0,
(q1,1)->q0};q0;F={q1}
ДКА, который принимает только строки нечётной длины
Ожидается: False
Получено: False
-----
ДКА: Q={q0};Σ={0,1};δ={(q0,0)->q0, (q0,1)->q0};q0;F={q0}
ДКА, который принимает все строки
Ожидается: True
Подходящая строка: Пустая строка
Получено: True
-----
ДКА: Q={q0};Σ={0,1};δ={(q0,0)->q0, (q0,1)->q0};q0;F={}

```


ДКА, который не принимает ничего

Ожидается: False

Получено: False

ДКА: $Q=\{q_0, q_1\}; \Sigma=\{0, 1\}; \delta=\{(q_0, 0) \rightarrow q_0, (q_0, 1) \rightarrow q_1, (q_1, 0) \rightarrow q_1, (q_1, 1) \rightarrow q_0\}; q_0; F=\{q_1\}$

ДКА, который принимает строки с нечётным количеством единиц

Ожидается: True

Подходящая строка: 01

Получено: True

ДКА: $Q=\{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}; \Sigma=\{0, 1\}; \delta=\{(q_0, 0) \rightarrow q_1, (q_0, 1) \rightarrow q_1, (q_1, 0) \rightarrow q_2, (q_1, 1) \rightarrow q_2, (q_2, 0) \rightarrow q_3, (q_2, 1) \rightarrow q_3, (q_3, 0) \rightarrow q_4, (q_3, 1) \rightarrow q_4, (q_4, 0) \rightarrow q_6, (q_4, 1) \rightarrow q_5, (q_5, 0) \rightarrow q_5, (q_5, 1) \rightarrow q_5, (q_6, 0) \rightarrow q_6, (q_6, 1) \rightarrow q_6\}; q_0; F=\{q_5\}$

ДКА, который принимает строки с 1 на 5й позиции

Ожидается: True

Подходящая строка: 000010

Получено: True

ДКА: $Q=\{q_0, q_1, q_2, q_3, q_4, q_5\}; \Sigma=\{0, 1\}; \delta=\{(q_0, 0) \rightarrow q_1, (q_0, 1) \rightarrow q_1, (q_1, 0) \rightarrow q_2, (q_1, 1) \rightarrow q_2, (q_2, 0) \rightarrow q_3, (q_2, 1) \rightarrow q_3, (q_3, 0) \rightarrow q_5, (q_3, 1) \rightarrow q_4, (q_4, 0) \rightarrow q_4, (q_4, 1) \rightarrow q_4, (q_5, 0) \rightarrow q_5, (q_5, 1) \rightarrow q_5\}; q_0; F=\{q_4\}$

ДКА, который принимает строки с 1 на 4й позиции

Ожидается: True

Подходящая строка: 0001

Получено: True

ДКА: $Q=\{q_0, q_1, q_2, q_3\}; \Sigma=\{0, 1\}; \delta=\{(q_0, 0) \rightarrow q_1, (q_0, 1) \rightarrow q_1, (q_1, 0) \rightarrow q_2, (q_1, 1) \rightarrow q_2, (q_2, 0) \rightarrow q_3, (q_2, 1) \rightarrow q_3, (q_3, 0) \rightarrow q_1, (q_3, 1) \rightarrow q_1\}; q_0; F=\{q_3\}$

Ожидается:

Подходящая строка: 000000

Получено: True

Вывод

В результате выполнения лабораторной работы для заданного языка, в котором предполагается, что ДКА определён над алфавитом $\{0, 1\}$, была построено описание МТ, решающей его, а также была реализована данная МТ в виде программы при условии, что $A = \{\langle M \rangle: M - \text{ДКА, который допускает хотя бы одну строку чётной длины.}\}$